Contents

Seasonal Time-Series Forecasting Research System 5
End-to-End Engineering Blueprint 5
1. Executive Summary 5
1.1 System purpose 5
1.2 System objectives 5
1.3 Out of scope 6
2. Research Design 6
2.1 Experimental variables 6
2.2 Research hypotheses and interpretation 6
2.3 Synthetic data generation 7
Ground truth 7
2.4 Dataset generation rules 7
2.5 Train/test protocol 7
2.6 Forecasting methods 8
Method A: Classical decomposition 8
Method B: Holt-Winters 8
Method C: SARIMA 8
2.7 Evaluation metrics 8
3. System Requirements 9
3.1 Functional requirements 9
3.2 Non-functional requirements 9
Reproducibility 9
Performance and resource management 10
Reliability 10
Maintainability 10
Usability 10
Portability 10
4. High-Level Architecture 11
4.1 Architectural style 11
4.2 Architecture diagram 11
4.3 Component responsibilities 11
1. Configuration Manager 11
2. Dataset Generator 11
3. Experiment Manager / Orchestrator 12
4. Forecasting Engine 12
5. Evaluation Engine 12
6. Results Manager / Persistence Layer 12
7. Analysis Engine 12
8. CLI 13
9. GUI 13
10. Progress Reporting 13
11. Execution Strategy 13
5. Core Data Model 13
5.1 Dataset Batch 13
5.2 Dataset 14
5.3 Experiment 14
5.4 Model Evaluation 14
5.5 Entity relationship diagram 16

6. Persistence Design 17
6.1 Storage technologies 17
6.2 Why two storage mechanisms? 17
6.3 Workspace structure 17
6.4 Artifact integrity 18
6.5 SQLite integrity 18
7. Forecasting Interface and Evaluation Contracts 18
7.1 Common forecasting interface 18
7.2 Evaluation contract 19
7.3 Fair-comparison invariants 19
8. Experiment Execution and Orchestration 20
8.1 Experiment workflow 20
8.2 Supported workflow modes 22
Mode A: Generation only 22
Mode B: Generation and training 22
Mode C: Existing datasets 22
Mode D: Resume 22
8.3 Evaluation task independence 22
MVP execution strategy 22
Future execution strategy 23
9. Execution State and Recovery 23
9.1 Experiment states 23
9.2 Evaluation states 23
9.3 Resume behavior 23
9.4 Failure isolation 24
10. Progress Reporting and Logging 24
10.1 Three levels of progress 24
Level A: Experiment overview 24
Level B: Evaluation table 24
Level C: Current operation 24
10.2 Overall progress calculation 25
10.3 Progress event structure 25
10.4 Durable and live progress 25
10.5 Logging 26
11. Analysis and Results 26
11.1 Individual evaluation results 26
11.2 Grouped analysis 26
11.3 Best method per group 27
11.4 Visualizations 27
11.5 Export 27
12. User Interfaces 27
12.1 CLI 27
12.2 GUI 28
12.3 Shared workflow architecture 28
13. Configuration Design 29
13.1 Configuration format 29
13.2 Configuration principles 29
13.3 Configuration examples 29
14. Technology Stack 30
14.1 Explicitly excluded from MVP 30

15. Project Structure 31
16. Development Strategy 32
Phase 1 — Foundation 32
Phase 2 — Dataset Generator 33
Phase 3 — Forecasting Engine 33
Phase 4 — Evaluation and Persistence 34
Phase 5 — Experiment Orchestration 34
Phase 6 — Analysis and CLI 34
Phase 7 — GUI and Polish 35
17. Testing Strategy 35
17.1 Unit tests 35
17.2 Integration tests 35
17.3 System tests 36
17.4 Research-validity tests 36
Data leakage 36
Fair comparison 36
Parameter selection 36
Metric correctness 36
Reproducibility 36
Persistence and recovery 36
17.5 Test execution policy 36
18. Deployment, Delivery, and Collaboration 37
18.1 Deployment model 37
18.2 Installation 37
18.3 Version control 37
18.4 Continuous integration 37
18.5 Data sharing 37
19. Operational Safety and Maintenance 38
19.1 Resource management 38
19.2 Backup and integrity 38
19.3 Error handling 38
19.4 Operational observability priorities 38
20. Key Architectural Decisions 39
21. Critical Design Constraints and Invariants 40
Research integrity 40
Data integrity 40
Execution integrity 40
Architectural integrity 40
Operational integrity 40
22. Two-Day Implementation Plan 40
Day 1 — End-to-end foundation 40
Day 2 — Complete the research workflow 41
23. Definition of Done 41
Research 41
Persistence 42
Execution 42
Analysis 42
Delivery 42
Research validity 42
24. Open Implementation Specifications 42

25. Final Engineering Principle 43

Seasonal Time-Series Forecasting Research System

End-to-End Engineering Blueprint

Document type: System Design & Implementation Blueprint
Status: Approved design — ready for implementation
Version: 1.0
Primary implementation language: Python
Target environment: Linux and Windows
Project type: Local experimental research application
Primary objective: Compare seasonal forecasting methods under controlled synthetic time-series conditions.

---

1. Executive Summary

1.1 System purpose

The system is an experimental framework for evaluating and comparing three forecasting approaches:

1. Classical seasonal decomposition with an explicit forecasting procedure.
2. Holt-Winters triple exponential smoothing.
3. Seasonal ARIMA (SARIMA).

It generates synthetic seasonal time series with controlled characteristics, executes the forecasting methods under consistent experimental conditions, evaluates their predictions, and stores structured results for analysis.

The primary research question is:

> Which forecasting method performs better under different combinations of time-series characteristics?

The research does not seek to identify one universally superior forecasting method. Instead, it investigates how model performance changes with seasonality type, seasonal period, trend, noise level, and dataset length.

1.2 System objectives

The system must:

- Generate reproducible synthetic datasets.
- Preserve the underlying generated components as ground truth.
- Apply multiple forecasting methods to identical training and testing conditions.
- Prevent information leakage during fitting and parameter selection.
- Evaluate forecasts using consistent metrics.
- Store datasets, experiment configurations, forecasts, metrics, and execution metadata.
- Support individual and batch experiments.
- Resume interrupted experiments without repeating completed evaluations.
- Provide grouped analysis of forecasting performance.
- Expose the same research workflows through a CLI and an optional local GUI.
- Make execution progress and failures visible throughout the experiment lifecycle.

1.3 Out of scope

The MVP is not intended to be:

- A production forecasting API.
- A distributed computing platform.
- A cloud-hosted application.
- A multi-user server.
- A general-purpose AutoML platform.
- A real-time forecasting service.

Real-world datasets, distributed execution, advanced dashboards, and production deployment are future extensions.

---

2. Research Design

The research design must be fixed before implementation because it determines what the generator produces, what the models receive, and how results are interpreted.

2.1 Experimental variables

The synthetic generator will support the following dimensions:

Dimension	Values	Number of values
Seasonality type	Additive, multiplicative	2
Seasonal period	4, 7, 12	3
Trend	None, linear	2
Noise level	Low, medium, high	3
Dataset length	100, 200, 500	3
Replications	5 per combination	5

The number of unique parameter combinations is:

`2 × 3 × 2 × 3 × 3 = 108`

With five replications per combination:

`108 × 5 = 540 datasets`

With three forecasting methods:

`540 × 3 = 1,620 model evaluations`

These figures define the planned full experiment. They are not a requirement to execute the entire experiment during development or continuous integration.

2.2 Research hypotheses and interpretation

The framework should enable investigation of questions such as:

- Does additive decomposition perform differently from multiplicative decomposition?
- Does a forecasting method become more reliable as the dataset length increases?
- How does noise affect each method?
- Does a method behave differently when a linear trend is present?
- Does the seasonal period influence the relative performance of the methods?

The system should report observed performance, not hard-code expected winners or assume that a particular method must perform best.

2.3 Synthetic data generation

The generator produces an observed time series from its underlying components.

Conceptually:

Additive model

```
Y(t) = Trend(t) + Seasonality(t) + Noise(t)
```

Multiplicative model

```
Y(t) = Trend(t) × Seasonality(t) × NoiseFactor(t)
```

For multiplicative generation, factors must remain positive. For example, noise should be represented by a positive factor such as `1 + ε`, rather than adding unrestricted zero-centred noise directly to the final series.

The exact mathematical definitions, parameter ranges, and noise distributions must be specified in the generator configuration and documented. They must not change silently between experiments.

Ground truth

For every generated dataset, preserve:

- Observed series.
- Trend component.
- Seasonal component.
- Noise component or noise factor.
- Generator configuration.
- Random seed.
- Dataset metadata.

Ground truth is available for research analysis and validation. Forecasting methods must never receive ground-truth components as inputs.

2.4 Dataset generation rules

- Each replication receives a reproducible seed.
- A dataset has a unique identity.
- Generated datasets are immutable.
- Modifying a dataset creates a new dataset identity.
- Dataset metadata records every parameter needed to reproduce its generation.
- The system must not require all 540 datasets to remain in memory simultaneously.

2.5 Train/test protocol

Use chronological splitting.

- Training fraction: 80%.
- Testing fraction: 20%.
- Forecast horizon: configurable.
- Default horizon: one complete seasonal cycle.

For a dataset of length 100 and seasonal period 12:

- Training observations: 80.

- Testing observations: 20.
- Evaluation horizon: first 12 testing observations.
- Remaining 8 testing observations are not included in that evaluation.

All methods must receive the same training observations and predict the same horizon.

The test observations must not be used for:

- Parameter selection.
- Model fitting.
- Trend estimation.
- Seasonal estimation.
- Any preprocessing that learns from data.

2.6 Forecasting methods

Method A: Classical decomposition

This is a forecasting procedure, not merely a decomposition visualization.

The method must:

1. Estimate trend using training observations only.
2. Estimate seasonal effects using training observations only.
3. Define an explicit trend extrapolation procedure.
4. Repeat the estimated seasonal cycle into the future.
5. Reconstruct the forecast using the appropriate additive or multiplicative formulation.

The exact estimation and extrapolation rules must be documented and tested. They must not be left ambiguous.

Method B: Holt-Winters

Use `statsmodels` exponential smoothing functionality.

The method must support the relevant seasonal formulation and seasonal period. Its configuration and selected parameters must be recorded.

Method C: SARIMA

Use `statsmodels` SARIMAX functionality with a bounded automatic parameter-selection procedure.

The search must be configurable and constrained to prevent excessive computation.

Record:

- Search limits.
- Candidate configurations considered, where practical.
- Selected order.
- Selected seasonal order.
- Selection criterion.
- Any fitting warnings or failures.

Automatic selection must use training observations only.

2.7 Evaluation metrics

Calculate:

- MAE — Mean Absolute Error.
- MSE — Mean Squared Error.

- RMSE — Root Mean Squared Error.
- MAPE — Mean Absolute Percentage Error.

MAPE requires special care when actual values are zero or close to zero. Its implementation must define and test a clear policy for such values. Undefined or misleading MAPE values must not silently become valid-looking results.

Preserve the actual test observations and predicted values alongside the metrics.

---

3. System Requirements

3.1 Functional requirements

ID	Requirement
FR-01	Load and validate experiment configurations.
FR-02	Generate synthetic datasets from configured combinations.
FR-03	Generate reproducible datasets using controlled seeds.
FR-04	Preserve generated ground-truth components.
FR-05	Persist and retrieve datasets.
FR-06	Execute the three forecasting methods through a common interface.
FR-07	Automatically select model parameters within configured limits.
FR-08	Apply a common chronological train/test protocol.
FR-09	Calculate evaluation metrics consistently.
FR-10	Persist forecasts, metrics, parameters, and execution statuses.
FR-11	Run one evaluation, one experiment, or a batch experiment.
FR-12	Run forecasting on previously generated datasets.
FR-13	Resume interrupted experiments.
FR-14	Skip evaluations already completed successfully.
FR-15	Continue other evaluations when an individual model evaluation fails.
FR-16	Aggregate results by experimental characteristics.
FR-17	Identify the best-performing method within each group and metric.
FR-18	Export experiment results.
FR-19	Display progress and execution status.
FR-20	Provide CLI and GUI access to shared application workflows.

3.2 Non-functional requirements

Reproducibility

The same generator configuration and seed must produce identical observations and ground-truth components.

Record:

- Random seed.
- Generator configuration.

- Experiment configuration.
- Model configuration.
- Selected model parameters.
- Python version.
- Relevant library versions.

Exact bit-for-bit equality of model forecasts across different platforms or library versions is not guaranteed.

Performance and resource management

Target hardware:

- Intel Core i5, 6th generation.
- 8 GB RAM.

Requirements:

- Sequential execution is the MVP default.
- Load datasets on demand.
- Avoid keeping the entire experiment in memory.
- Avoid unnecessary data copies.
- Bound SARIMA parameter search.
- Release temporary objects after evaluations.
- Measure runtime before setting performance targets.

Reliability

- Persist completed evaluations.
- Record failed evaluations and their errors.
- Resume interrupted experiments.
- Do not mark an evaluation complete before its required results are safely persisted.
- Stop safely when persistence becomes unreliable.

Maintainability

- Clear component boundaries.
- Common forecasting interface.
- Independent storage responsibilities.
- Configuration validation before execution.
- Adding a forecasting method must not require rewriting the experiment manager.

Usability

- CLI is the primary interface.
- GUI is optional and local.
- Clear progress reporting.
- Human-readable errors.
- Results should be inspectable without manually navigating artifact files.

Portability

The application must work on supported Linux and Windows environments.

Use portable filesystem operations, avoid hard-coded operating-system paths, and validate installation and execution on both platforms.

---

4. High-Level Architecture

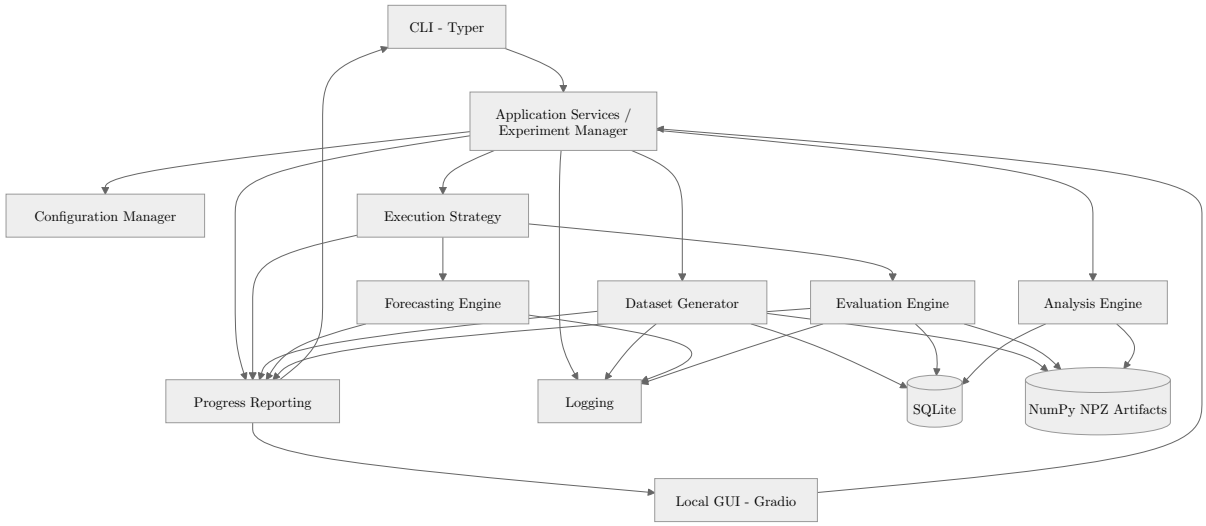
4.1 Architectural style

Modular monolith.

All components run within one Python application. They communicate through direct Python interfaces and structured inputs/outputs.

There will be no internal HTTP services, message brokers, microservices, or distributed execution infrastructure in the MVP.

4.2 Architecture diagram



4.3 Component responsibilities

1. Configuration Manager

Responsibilities:

- Load YAML configuration.
- Validate configuration using Pydantic.
- Apply defaults.
- Reject invalid configurations before execution.
- Provide validated configuration objects to application services.

It must not execute experiments or implement forecasting logic.

2. Dataset Generator

Responsibilities:

- Generate individual synthetic time series.
- Generate combinations and replications.
- Preserve ground-truth components.
- Assign seeds and dataset IDs.
- Produce dataset metadata.
- Persist generated datasets.

It must not train forecasting models.

3. Experiment Manager / Orchestrator

Responsibilities:

- Create and manage experiment records.
- Build the experiment execution plan.
- Coordinate generation and evaluation.
- Schedule independent evaluation tasks through the execution strategy.
- Track experiment status.
- Handle resumability.
- Coordinate progress reporting.

It coordinates work; it does not implement model fitting, metric calculation, generation mathematics, or storage internals.

4. Forecasting Engine

Responsibilities:

- Expose a common forecasting interface.
- Register and instantiate forecasting methods.
- Fit models using training observations.
- Perform parameter selection.
- Generate forecasts for a requested horizon.
- Return structured results and model metadata.

It must not decide how an entire experiment is scheduled.

5. Evaluation Engine

Responsibilities:

- Prepare the chronological train/test split.
- Enforce the configured forecast horizon.
- Execute the model evaluation protocol.
- Calculate metrics.
- Validate forecasts.
- Produce structured evaluation results.

It must not independently implement experiment orchestration.

6. Results Manager / Persistence Layer

Responsibilities:

- Store and retrieve experiments.
- Store and retrieve datasets and evaluations.
- Manage database transactions.
- Coordinate numerical artifact persistence.
- Validate artifact references.
- Support recovery and integrity checks.

It owns persistence details. Other components should not issue arbitrary SQL or manipulate artifact files independently.

7. Analysis Engine

Responsibilities:

- Retrieve completed evaluation results.
- Group results by experimental characteristics.

- Aggregate metrics.
- Compare methods.
- Identify the best method within each group and metric.
- Prepare exportable result tables.

It must not change original evaluation results.

8. CLI

Responsibilities:

- Parse user commands and options.
- Invoke application services.
- Display progress, statuses, errors, and results.
- Support configuration and workspace selection.

It must not contain independent implementations of research workflows.

9. GUI

Responsibilities:

- Provide local graphical access to the same application operations.
- Display experiment status.
- Display evaluation-level progress.
- Display current execution stage.
- Show result tables and visualizations.

It must not duplicate orchestration or evaluation logic.

10. Progress Reporting

Responsibilities:

- Receive structured progress events.
- Make live events available to the CLI and GUI.
- Maintain consistent progress semantics.
- Persist meaningful state transitions through the relevant application services.

Progress reporting must be shared across interfaces.

11. Execution Strategy

Responsibilities:

- Define how independent evaluation tasks are executed.
- Provide a sequential implementation for the MVP.
- Provide a stable contract that can support future parallel execution.

The experiment manager must depend on the execution strategy interface rather than hard-code sequential loops throughout the application.

---

5. Core Data Model

The system has four primary conceptual entities.

5.1 Dataset Batch

A Dataset Batch groups datasets generated together under a common generation request.

Example:

- Batch ID: `batch_001`
- Generation configuration: full factorial experiment.
- Dataset count: 540.
- Status: completed.

A batch is a logical grouping, not a single large dataset file.

5.2 Dataset

Represents one immutable generated time series.

Conceptual attributes:

- Dataset ID.
- Batch ID.
- Generation configuration.
- Random seed.
- Seasonality type.
- Seasonal period.
- Trend type.
- Noise level.
- Dataset length.
- Observations artifact reference.
- Ground-truth artifact reference.
- Creation timestamp.
- Integrity information.

Observed data and ground truth must be logically separated.

5.3 Experiment

Represents a research execution request.

An experiment can reference:

- A generated batch.
- Selected existing datasets.
- One or more forecasting methods.
- Evaluation configuration.
- Experiment configuration.
- Execution status.
- Creation and completion timestamps.

An experiment should not own copies of every dataset. It references datasets by ID.

5.4 Model Evaluation

Represents one model applied to one dataset under one experiment's evaluation configuration.

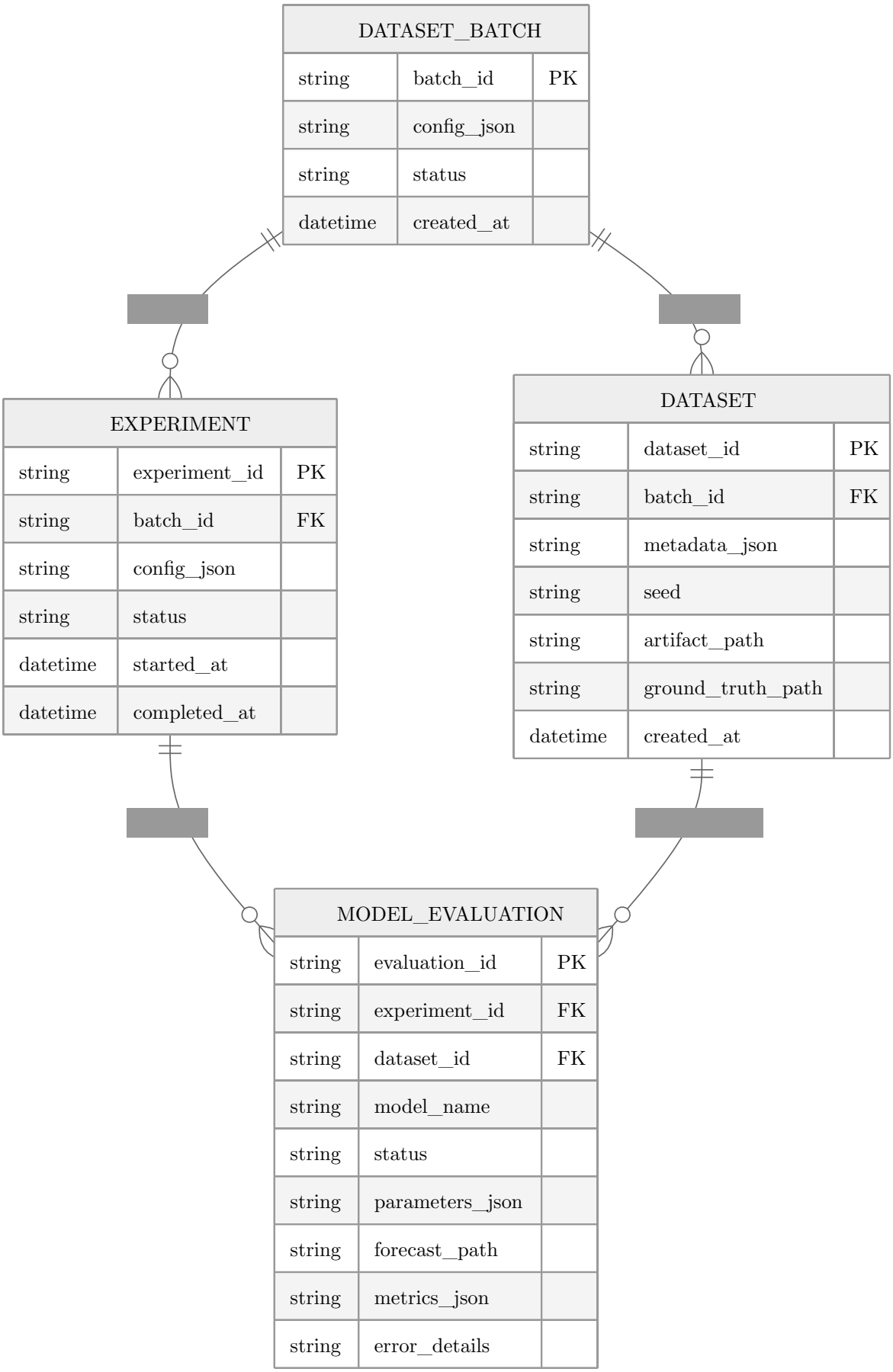
Attributes:

- Evaluation ID.
- Experiment ID.
- Dataset ID.
- Model name.
- Evaluation status.
- Train/test split metadata.
- Forecast horizon.

- Selected parameters.
- Actual test observations artifact.
- Forecast artifact.
- Metrics.
- Runtime information.
- Error details, if applicable.
- Timestamps.

A model evaluation is the unit of independent execution and recovery.

5.5 Entity relationship diagram



This diagram describes logical relationships. The physical SQLite schema may normalize metrics and parameters into separate tables where that improves querying and integrity.

---

6. Persistence Design

6.1 Storage technologies

Data	Storage
Experiments	SQLite
Dataset batches	SQLite
Dataset metadata	SQLite
Dataset observations	NumPy NPZ
Ground-truth components	NumPy NPZ
Evaluation metadata	SQLite
Selected model parameters	SQLite
Metrics	SQLite
Actual test observations	NumPy NPZ
Forecast arrays	NumPy NPZ
Logs	Text log files

6.2 Why two storage mechanisms?

SQLite is suitable for structured records, relationships, statuses, metadata, and analytical queries.

NumPy NPZ is suitable for numerical arrays such as observations, components, actual values, and predictions.

This avoids storing large numerical arrays as database blobs or forcing the application to reconstruct arrays from database records.

6.3 Workspace structure

Keep runtime data outside the source package.

```
workspace/
├── database/
│   └── research.sqlite
│
├── artifacts/
│   ├── datasets/
│   │   └── <dataset_id>.npz
│   │
│   └── evaluations/
│       └── <evaluation_id>.npz
│
├── logs/
│   ├── application.log
│   └── experiments/
│       └── <experiment_id>.log
│
├── exports/
```

```
│
└── backups/
```

The workspace location must be configurable.

6.4 Artifact integrity

The persistence layer must ensure that:

- Artifact writes are completed before their references are committed.
- Temporary files are not treated as valid artifacts.
- Final artifact writes use an atomic replacement strategy where supported.
- Database records do not point to incomplete artifacts.
- Artifact paths are stored consistently and portably.
- Missing or corrupt artifacts are detected.
- Dataset artifacts are not silently overwritten.

A safe persistence sequence is:

1. Write an artifact to a temporary path.
2. Validate that it can be read.
3. Move it to its final path using an atomic replacement operation where supported.
4. Commit the corresponding database reference.
5. If the database transaction fails, leave the artifact as an identifiable orphan for later cleanup rather than creating a false database reference.

For evaluation results, only mark the evaluation `COMPLETED` after all required artifacts and metadata have been safely persisted.

6.5 SQLite integrity

Use transactions for related database updates.

The persistence layer should support:

- Foreign-key constraints.
- Unique IDs.
- Appropriate indexes.
- Transactional status updates.
- SQLite integrity checks.
- A safe backup mechanism.

Do not copy a live SQLite database file naively as a backup. Use SQLite's supported backup mechanism.

---

7. Forecasting Interface and Evaluation Contracts

7.1 Common forecasting interface

All forecasting methods must implement the same conceptual contract.

Inputs:

- Training observations.
- Seasonal period.
- Forecast horizon.
- Model-specific configuration.

- Optional metadata required by the method.

Outputs:

- Forecast array.
- Selected parameters.
- Model metadata.
- Execution information.

Requirements:

- No access to test observations.
- No access to ground truth.
- Forecast length must equal the requested horizon.
- Forecast values must be finite.
- Invalid input must produce a clear error.
- Model failures must be identifiable and recordable.

A method registry should allow methods to be discovered and instantiated by a stable model name.

7.2 Evaluation contract

Every evaluation follows the same logical protocol:

1. Load one dataset.
2. Validate the dataset.
3. Prepare the chronological split.
4. Determine the evaluation horizon.
5. Give only training observations to the forecasting method.
6. Select parameters using training data only.
7. Fit the model.
8. Forecast the configured horizon.
9. Validate forecast shape and values.
10. Retrieve the corresponding actual test observations.
11. Calculate metrics.
12. Persist the actual values, forecast, parameters, metrics, and execution metadata.
13. Mark the evaluation complete.

7.3 Fair-comparison invariants

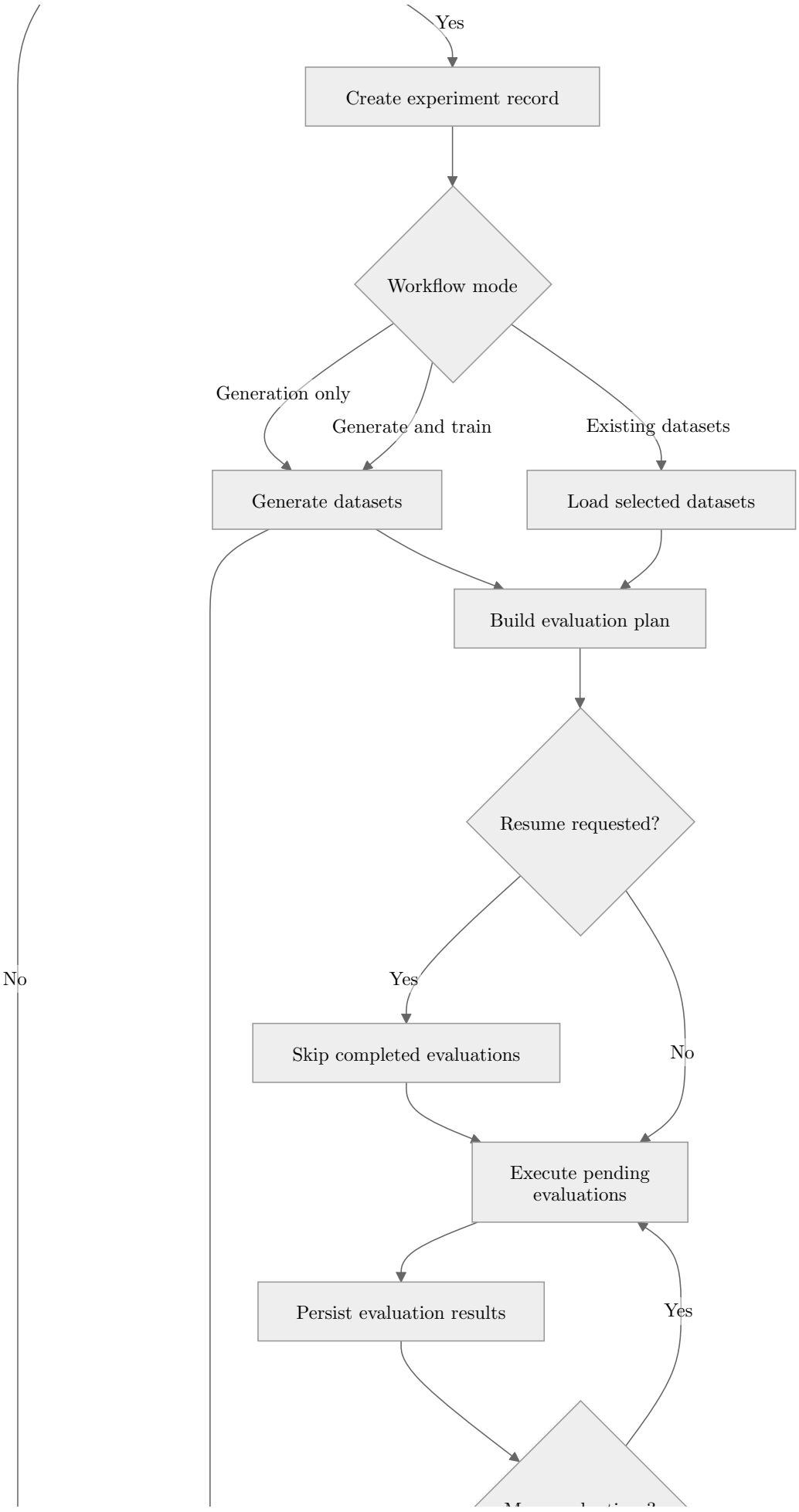
The following are mandatory research-validity rules:

- Every model receives identical training observations.
- Every model predicts the same horizon.
- Every model is evaluated against identical actual observations.
- No model receives ground-truth components.
- No model receives future test observations during fitting or selection.
- Metrics use the same implementation and definitions.
- Failed evaluations remain distinguishable from valid evaluations.
- Comparisons must not silently omit failures without reporting them.

---

8. Experiment Execution and Orchestration

8.1 Experiment workflow


Yes
Create experiment record
Workflow mode
Generation only
Generate and train
Existing datasets
Generate datasets
Load selected datasets
Build evaluation plan
Resume requested?
No
Yes
Skip completed evaluations
No
Execute pending evaluations
Persist evaluation results
Yes

8.2 Supported workflow modes

Mode A: Generation only

- Validate configuration.
- Create a batch.
- Generate datasets.
- Persist datasets and metadata.
- Mark the batch complete.

Mode B: Generation and training

- Generate datasets.
- Create an experiment.
- Build the model evaluation plan.
- Execute all required evaluations.
- Persist results.
- Optionally perform analysis.

Mode C: Existing datasets

- Select previously generated datasets.
- Create an experiment.
- Select forecasting methods.
- Execute evaluations.
- Persist and analyze results.

Mode D: Resume

- Load the experiment.
- Inspect all evaluation records.
- Skip completed evaluations.
- Identify pending, interrupted, and recoverable failed evaluations.
- Execute eligible evaluations.
- Update the experiment status.

The resume policy must distinguish retryable failures from permanent failures. A retry should not overwrite the diagnostic history of the earlier attempt.

8.3 Evaluation task independence

An evaluation task is defined by:

- Experiment ID.
- Dataset ID.
- Model name.
- Evaluation configuration.

Each task must be independently identifiable and persistable.

The execution strategy processes these tasks.

MVP execution strategy

Sequential execution:

- One evaluation at a time.
- Predictable memory usage.
- Easier debugging.

- Simpler persistence coordination.
- Easier recovery.

Future execution strategy

Parallel execution is deferred.

The architecture must allow a future implementation to execute independent tasks concurrently without rewriting the experiment manager or forecasting methods.

Parallel execution must introduce explicit resource limits and safe persistence coordination.

---

9. Execution State and Recovery

9.1 Experiment states

Use explicit experiment states:

- `CREATED`
- `RUNNING`
- `COMPLETED`
- `FAILED`
- `INTERRUPTED`

A partially successful experiment should not be incorrectly reported as fully successful.

Its final status should reflect the actual outcome, while individual evaluation statuses preserve detailed information.

9.2 Evaluation states

Use explicit evaluation states:

- `PENDING`
- `RUNNING`
- `COMPLETED`
- `FAILED`
- `INTERRUPTED`

The exact recovery transition policy must be centralized rather than scattered across interfaces.

9.3 Resume behavior

On resume:

1. Load the experiment and its configuration.
2. Load evaluation records.
3. Verify which completed evaluations have valid required artifacts.
4. Skip valid completed evaluations.
5. Identify interrupted or retryable failed evaluations.
6. Re-run eligible tasks.
7. Preserve prior attempt diagnostics.
8. Recalculate experiment-level status.

A completed evaluation with missing or corrupt artifacts must not be treated as valid merely because its database status says `COMPLETED`.

9.4 Failure isolation

A forecasting failure should normally affect only its own evaluation.

For example:

- SARIMA fails on one dataset.
- Its evaluation is recorded as failed.
- The next evaluation proceeds.
- The experiment reports the failure.
- The user can inspect the error and retry if appropriate.

A database failure is different. If reliable persistence cannot be guaranteed, execution must stop safely rather than continue producing results that cannot be recorded.

---

10. Progress Reporting and Logging

Progress visibility is a core requirement, not a cosmetic feature.

The user should never have to guess whether the system is working, frozen, or waiting.

10.1 Three levels of progress

Level A: Experiment overview

Display:

- Experiment ID.
- Overall status.
- Total evaluations.
- Completed evaluations.
- Failed evaluations.
- Running evaluations.
- Pending evaluations.
- Overall progress percentage.
- Current activity.
- Elapsed time.
- Approximate ETA when sufficient timing history exists.

Level B: Evaluation table

Display one row per evaluation:

- Dataset ID.
- Model name.
- Evaluation ID.
- Status.
- Start time.
- Duration.
- Current stage.
- Error summary, if failed.

Level C: Current operation

Display the current evaluation stage:

1. Load dataset.
2. Prepare split.

3. Select parameters.
4. Fit model.
5. Generate forecast.
6. Calculate metrics.
7. Persist results.
8. Complete.

Do not invent internal progress percentages for operations whose progress cannot be measured reliably. Display their current stage instead.

10.2 Overall progress calculation

Use:

```
Progress = (completed evaluations + failed evaluations) / total evaluations × 100
```

Failed evaluations count as processed work but remain visibly separate from successful evaluations.

Pending and running counts must also be displayed.

ETA should be approximate and only shown when enough timing history exists to support a reasonable estimate.

10.3 Progress event structure

A progress event should contain:

- Experiment ID.
- Evaluation ID, when applicable.
- Dataset ID, when applicable.
- Model name, when applicable.
- Current stage.
- Status.
- Timestamp.
- Human-readable message.

The event mechanism must be shared by CLI and GUI.

The execution components publish events. The interfaces subscribe and render them.

10.4 Durable and live progress

Not every live event needs to be stored in SQLite.

Persist meaningful transitions such as:

- Experiment created.
- Experiment started.
- Evaluation started.
- Evaluation stage changed when operationally important.
- Evaluation completed.
- Evaluation failed.
- Experiment interrupted.
- Experiment completed.

Use live events for more frequent interface updates.

10.5 Logging

Use Python's standard `logging` module.

Log destinations:

- Console.
- Application-wide log file.
- Per-experiment log file.

Log levels:

- DEBUG.
- INFO.
- WARNING.
- ERROR.
- CRITICAL.

Every relevant log entry should include available identifiers, especially experiment ID, evaluation ID, dataset ID, and model name.

Avoid logging entire numerical arrays or unnecessarily large objects.

Logs provide diagnostic detail; database records provide authoritative execution state.

---

11. Analysis and Results

11.1 Individual evaluation results

For every completed evaluation, preserve:

- Dataset identity.
- Experiment identity.
- Model name.
- Actual values.
- Predicted values.
- Forecast horizon.
- Selected parameters.
- Metrics.
- Runtime.
- Relevant warnings.

11.2 Grouped analysis

The analysis engine must support grouping by experimental dimensions:

- Seasonality type.
- Seasonal period.
- Trend.
- Noise level.
- Dataset length.

It should aggregate metrics for each model within each group.

For example:

Seasonality	Period	Trend	Noise	Length	Model	Mean RMSE
Additive	12	Linear	Low	200	Method A	…
Additive	12	Linear	Low	200	Method B	…
Additive	12	Linear	Low	200	Method C	…

The table is illustrative; actual values must come from stored evaluations.

11.3 Best method per group

Identify the best method separately for each selected metric.

For example:

- Best method by mean RMSE.
- Best method by mean MAE.
- Best method by mean MAPE.

Do not assume that the same method wins across all metrics.

Where useful, also report:

- Number of successful evaluations.
- Number of failed evaluations.
- Standard deviation.
- Median.
- Minimum and maximum.

The aggregation policy must be explicit, particularly when some evaluations failed.

11.4 Visualizations

Secondary MVP priority:

- Actual versus predicted values.
- Grouped metric comparisons.
- Model performance across seasonal periods.
- Model performance across noise levels.
- Model performance across dataset lengths.

Use Matplotlib.

Visualizations must be generated from stored results, not from hidden in-memory state.

11.5 Export

Support export of structured result tables.

CSV is the primary export format for MVP. Additional formats can be added later.

---

12. User Interfaces

12.1 CLI

The CLI is the primary interface.

Use Typer.

Conceptual command groups:

```
research
├── generate
├── run
├── evaluate
├── resume
├── experiments
│   ├── list
│   ├── show
│   └── status
├── datasets
│   ├── list
│   └── show
├── analyze
├── export
└── workspace
```

The exact command names and argument structure should be finalized during implementation.

The CLI must support:

- Configuration file selection.
- Workspace selection.
- Individual and batch workflows.
- Existing dataset selection.
- Resume.
- Experiment inspection.
- Progress display.
- Result analysis.
- Export.

12.2 GUI

Use Gradio.

The GUI is a secondary local interface.

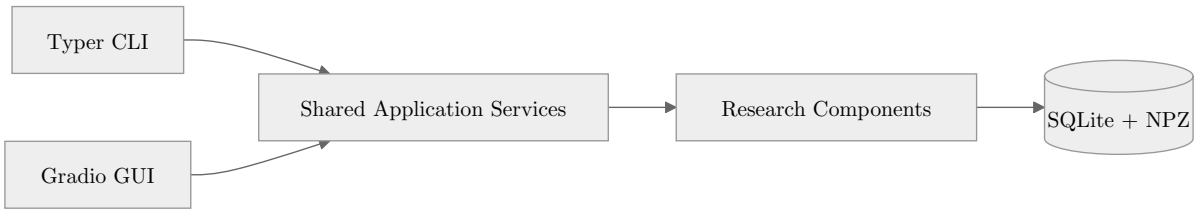
Conceptual pages or sections:

1. Experiment configuration.
2. Dataset generation.
3. Experiment execution.
4. Live experiment progress.
5. Evaluation details.
6. Result analysis.
7. Visualizations.
8. Export.

The GUI must call the same application services as the CLI.

It must not expose a separate implementation of the research workflow.

12.3 Shared workflow architecture



The CLI must remain usable even if Gradio is not installed or launched.

---

13. Configuration Design

13.1 Configuration format

Use YAML for user-facing configuration and Pydantic for validation.

Configuration should be divided into logical sections:

- Experiment.
- Generation.
- Evaluation.
- Forecasting methods.
- Execution.
- Workspace.
- Logging.

13.2 Configuration principles

- Validate before creating or executing an experiment.
- Define sensible defaults.
- Reject unsupported values.
- Keep model-specific settings separate from experiment-wide settings.
- Record the effective validated configuration with the experiment.
- Avoid hidden configuration changes during execution.

13.3 Configuration examples

The following is a conceptual structure, not a final validated schema:

```yaml
experiment:
  name: seasonal_forecasting_comparison
  description: Compare forecasting methods under controlled conditions

generation:
  seasonality_types:
    - additive
    - multiplicative

  seasonal_periods: [4, 7, 12]
  trends: [none, linear]
  noise_levels: [low, medium, high]
  dataset_lengths: [100, 200, 500]
  replications: 5
  base_seed: 42

evaluation:
  train_fraction: 0.8
  forecast_horizon: null
  default_horizon: one_seasonal_cycle

forecasting:
  methods:
    - classical_decomposition
    - holt_winters
    - sarima

  sarima:
    max_order: configurable
```

```yaml
    max_seasonal_order: configurable
    selection_criterion: aic

execution:
  strategy: sequential

workspace:
  path: ./workspace

logging:
  level: INFO
```

The final schema must define the exact allowed fields, types, constraints, and defaults. Placeholder values such as SARIMA search limits must be replaced with explicit bounded values before running the full experiment.

---

14. Technology Stack

Responsibility	Technology
Programming language	Python
Dependency management	uv
Dependency lockfile	`uv.lock`
Configuration	YAML
Configuration validation	Pydantic
Numerical computing	NumPy
Data manipulation	pandas
Forecasting	statsmodels
Classical decomposition	Custom NumPy/pandas implementation
Evaluation	NumPy / scikit-learn
Structured persistence	SQLite
Numerical artifacts	NumPy NPZ
CLI	Typer
GUI	Gradio
Visualization	Matplotlib
Testing	pytest
Version control	Git
Remote repository	GitHub
CI	GitHub Actions

14.1 Explicitly excluded from MVP

Do not introduce these technologies without a new design decision:

- MLflow.
- DVC.
- PostgreSQL.

- Docker.
- Celery.
- Redis.
- Distributed task frameworks.
- Prometheus.
- Grafana.
- Cloud infrastructure.

They do not solve a necessary MVP problem for this local research application.

---

15. Project Structure

Proposed repository layout:

```
seasonal-forecasting-research/
│
├── pyproject.toml
├── uv.lock
├── README.md
├── .gitignore
├── .python-version
│
├── configs/
│   ├── default.yaml
│   └── examples/
│
├── src/
│   └── forecasting_research/
│       │
│       ├── __init__.py
│       ├── main.py
│       │
│       ├── config/
│       │   ├── loader.py
│       │   ├── schemas.py
│       │   └── defaults.py
│       │
│       ├── domain/
│       │   ├── entities.py
│       │   ├── enums.py
│       │   ├── contracts.py
│       │   └── exceptions.py
│       │
│       ├── generation/
│       │   ├── generator.py
│       │   ├── components.py
│       │   └── seeds.py
│       │
│       ├── forecasting/
│       │   ├── base.py
│       │   ├── registry.py
│       │   ├── classical_decomposition.py
│       │   ├── holt_winters.py
│       │   └── sarima.py
│       │
│       ├── evaluation/
│       │   ├── splitter.py
│       │   ├── evaluator.py
│       │   └── metrics.py
│       │
│       ├── execution/
│       │   ├── base.py
│       │   └── sequential.py
```

```
│       │
│       ├── experiments/
│       │   ├── manager.py
│       │   ├── planner.py
│       │   ├── lifecycle.py
│       │   └── recovery.py
│       │
│       ├── persistence/
│       │   ├── database.py
│       │   ├── repositories/
│       │   ├── artifacts.py
│       │   └── backup.py
│       │
│       ├── analysis/
│       │   ├── aggregation.py
│       │   ├── comparison.py
│       │   └── export.py
│       │
│       ├── monitoring/
│       │   ├── events.py
│       │   ├── progress.py
│       │   └── logging.py
│       │
│       ├── application/
│       │   ├── services.py
│       │   └── use_cases/
│       │
│       ├── cli/
│       │   └── app.py
│       │
│       └── gui/
│           └── app.py
│
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── system/
│   └── research_validity/
│
├── docs/
│   ├── architecture.md
│   ├── research_methodology.md
│   └── decisions/
│
├── .github/
│   └── workflows/
│       └── ci.yml
│
└── workspace/
    └── .gitkeep
```

This is a proposed logical structure. It should be adjusted when implementation reveals that a package or module would be unnecessarily fragmented. Runtime workspace contents must not be committed.

---

16. Development Strategy

Build the application through incremental vertical slices.

Do not implement every module independently and wait until the end to integrate them.

Phase 1 — Foundation

Deliver:

- Repository structure.
- Python package setup.
- uv dependency management.
- Committed lockfile.
- Configuration loading.
- Pydantic validation.
- Logging foundation.
- Shared domain entities and contracts.
- Workspace configuration.

Acceptance criteria:

- Application environment installs successfully.
- Configuration loads and validates.
- Invalid configuration is rejected clearly.
- Shared contracts are defined and tested.
- Logging works.

Phase 2 — Dataset Generator

Deliver:

- Synthetic component generation.
- Additive and multiplicative generation.
- Trend generation.
- Noise levels.
- Seasonal periods.
- Dataset lengths.
- Replications and seeds.
- Ground-truth preservation.
- Dataset persistence and retrieval.

Acceptance criteria:

- Identical configuration and seed reproduce the same dataset.
- Ground-truth components are preserved.
- Dataset metadata is correct.
- Dataset artifacts can be saved and loaded.
- Dataset generation does not require loading the full batch into memory.

Phase 3 — Forecasting Engine

Deliver:

- Common forecasting interface.
- Method registry.
- Classical decomposition.
- Holt-Winters.
- Bounded SARIMA selection.

Implementation order:

1. Implement one method through the common interface.
2. Validate the interface and result contract.
3. Add the remaining methods.

Acceptance criteria:

- All methods receive the same type of training input.
- All return forecasts of the requested horizon.
- Forecasts contain valid finite values.
- Selected parameters are recorded.
- Failures are reported consistently.

Phase 4 — Evaluation and Persistence

Deliver:

- Chronological split.
- Configurable horizon.
- Metric calculations.
- Evaluation result contract.
- Evaluation persistence.
- Artifact integrity checks.

Acceptance criteria:

- Correct 80/20 split.
- Identical evaluation observations across methods.
- No test-data leakage.
- Correct metric calculations.
- Actual and predicted values are preserved.
- Results survive application restart.

Phase 5 — Experiment Orchestration

Deliver:

- Experiment creation.
- Dataset and model evaluation planning.
- Sequential execution.
- Progress events.
- Experiment lifecycle.
- Evaluation statuses.
- Failure isolation.
- Resume and recovery.

Acceptance criteria:

- A complete experiment can execute.
- Individual model failures do not unnecessarily stop other evaluations.
- Completed evaluations are skipped on resume.
- Interrupted work can recover.
- Progress is visible during execution.
- Persistence failures stop execution safely.

Phase 6 — Analysis and CLI

Deliver:

- Grouped metric aggregation.
- Model comparison.
- Best method per group and metric.
- CLI workflows.
- Result inspection.

- CSV export.

Acceptance criteria:

- Results can be analyzed from stored records.
- Grouping produces correct aggregates.
- Failed evaluations are reported.
- CLI workflows use shared application services.
- Exported results match stored results.

Phase 7 — GUI and Polish

Deliver:

- Local Gradio interface.
- Experiment configuration interface.
- Live progress display.
- Evaluation table.
- Result analysis.
- Visualizations.
- Documentation and installation instructions.

Acceptance criteria:

- GUI uses the same application services as the CLI.
- Long-running work does not make the interface appear frozen.
- Progress and failures remain visible.
- CLI works independently of GUI.
- Linux and Windows installation paths are documented and tested.

---

17. Testing Strategy

Testing is divided into four levels.

17.1 Unit tests

Test components independently:

- Configuration validation.
- Generator mathematics.
- Seed determinism.
- Dataset metadata.
- Forecast output shape.
- Forecast finiteness.
- Metric calculations.
- MAPE edge cases.
- Train/test splitting.
- Artifact roundtrips.
- Repository operations.
- Aggregation logic.
- State transitions.

17.2 Integration tests

Test interactions between components:

- Configuration → generator → persistence.
- Dataset → split → forecasting → evaluation.
- Evaluation → artifact storage → database.
- Experiment manager → execution strategy → evaluation.
- Analysis → persisted evaluation results.

17.3 System tests

Test complete workflows:

- Generation only.
- Single evaluation.
- Single experiment.
- Existing datasets.
- Batch experiment.
- Resume.
- Model failure.
- Persistence failure.
- Analysis.
- Export.

17.4 Research-validity tests

These tests are essential.

Data leakage

Verify that forecasting methods receive only training observations and do not access test values or ground-truth components.

Fair comparison

Verify that all methods receive identical training data and are evaluated on identical test observations and horizons.

Parameter selection

Verify that parameter selection does not use test observations.

Metric correctness

Verify metrics against known examples, including zero and near-zero actual values for MAPE.

Reproducibility

Verify that the same seed and generator configuration reproduce the same dataset.

Persistence and recovery

Verify that completed evaluations can be retrieved and that resume skips valid completed evaluations.

17.5 Test execution policy

Do not run the entire 1,620-evaluation experiment in every CI execution.

CI should run fast unit and integration tests. A representative research experiment should be run deliberately before the full batch.

---

18. Deployment, Delivery, and Collaboration

18.1 Deployment model

The application runs locally on the user's machine.

No cloud server, VPS, public API, or container orchestration is required.

18.2 Installation

Use:

- `pyproject.toml`.
- `uv.lock`.
- Documented installation instructions.
- A supported Python version.

Installation must be validated on Linux and Windows.

18.3 Version control

Use Git and GitHub.

The user is the primary developer. Teammates will install and use the application but will not contribute code or push to the repository.

Use branches and pull requests for development discipline where appropriate.

18.4 Continuous integration

GitHub Actions should:

1. Set up supported Python.
2. Install uv.
3. Install dependencies from the lockfile.
4. Run formatting and lint checks.
5. Run unit tests.
6. Run integration tests.
7. Validate package build.
8. Test installation and basic CLI startup on Linux and Windows.

GUI startup should also be checked where practical without making CI unnecessarily slow.

18.5 Data sharing

Do not commit large generated datasets, SQLite databases, or runtime artifacts to the source repository.

Share:

- Source code.
- Configuration files.
- Documentation.
- Small test fixtures.

Users maintain independent workspaces by default.

Results can be transferred through an intentional export or workspace-transfer process.

---

19. Operational Safety and Maintenance

19.1 Resource management

- Sequential execution is the default.
- Avoid loading all generated datasets at once.
- Load one dataset for an evaluation when possible.
- Release temporary numerical objects.
- Bound SARIMA search.
- Monitor memory and execution time during representative runs.

19.2 Backup and integrity

The system should provide:

- SQLite backup support.
- Artifact existence checks.
- Artifact readability checks.
- Database integrity checks.
- Missing-reference detection.
- Orphan artifact detection.

Do not automatically delete artifacts or previous experiment results.

19.3 Error handling

Failure	Expected behavior
Invalid configuration	Reject before execution
Dataset generation failure	Record failure and prevent invalid dataset use
Individual model failure	Record evaluation failure; continue eligible tasks
Invalid forecast	Fail evaluation with diagnostic information
Artifact write failure	Do not mark evaluation complete
Database failure	Stop safely
Application interruption	Preserve recoverable state
Missing artifact during resume	Flag integrity problem; do not blindly skip

19.4 Operational observability priorities

Must have:

- Experiment status.
- Evaluation statuses.
- Current operation stage.
- Completed, failed, running, and pending counts.
- Shared CLI/GUI progress events.
- Application and per-experiment logs.
- Failure diagnostics.
- Resume support.

Should have:

- Approximate ETA when meaningful.

- Workspace integrity checks.
- Database backup support.

Future:

- Advanced performance dashboards.
- Parallel worker monitoring.
- Distributed execution monitoring.

---

20. Key Architectural Decisions

This section records decisions that should not be changed casually during implementation.

Decision	Chosen approach	Reason
Architecture	Modular monolith	Appropriate for a local research tool
Communication	Direct Python calls	No need for internal network services
Primary interface	CLI	Reliable and easy to automate
Secondary interface	Gradio	Convenient visual access
Workflow ownership	Shared application services	Prevents duplicated business logic
Execution	Sequential MVP	Predictable resource usage
Future execution	Parallel-ready contract	Allows later extension
Structured storage	SQLite	Local, relational, transactional
Numerical storage	NumPy NPZ	Appropriate for numerical arrays
Configuration	YAML + Pydantic	Human-readable and validated
Dependency management	uv + lockfile	Reproducible installation
Forecasting library	statsmodels	Provides required forecasting methods
Classical decomposition	Custom implementation	Makes the forecasting procedure explicit
Testing	pytest	Supports unit through system tests
Deployment	Local application	Matches project requirements
Data sharing	Independent workspaces	Avoids committing large artifacts
Progress	Shared event mechanism	Consistent visibility across interfaces
Recovery	Evaluation-level resumability	Avoids repeating completed work

---

21. Critical Design Constraints and Invariants

The following constraints are mandatory throughout implementation.

Research integrity

1. Ground truth must remain separate from forecasting inputs.
2. No future test observations may influence model fitting or selection.
3. Every model must use the same train/test protocol.
4. Every model must be evaluated on the same forecast horizon.
5. Metric definitions must remain consistent.
6. Failed evaluations must not be silently represented as successful results.

Data integrity

7. Dataset identities must be unique.
8. Generated datasets are immutable.
9. Every evaluation must reference the correct experiment, dataset, and model.
10. A completed evaluation must have valid required results and artifacts.
11. Database and artifact references must be verifiable.

Execution integrity

12. Completed evaluations must be skipped during valid resume operations.
13. Individual model failures should be isolated.
14. Persistence failures must stop unsafe execution.
15. Experiment-level status must reflect the underlying evaluation outcomes.

Architectural integrity

16. CLI and GUI must use the same application services.
17. The experiment manager must coordinate, not implement every component's logic.
18. Forecasting methods must implement a common interface.
19. Execution strategy must be replaceable.
20. Parallel execution must not be introduced into the MVP without a deliberate design and resource-management decision.

Operational integrity

21. Progress must be visible.
22. Long-running operations must provide meaningful status updates.
23. Runtime artifacts must remain outside the source package.
24. Workspace data must not be automatically deleted.
25. Linux and Windows compatibility must be considered from the beginning.

---

22. Two-Day Implementation Plan

The two-day deadline requires strict MVP prioritization.

Day 1 — End-to-end foundation

Priority:

1. Project foundation.
2. Configuration and validation.

3. Shared contracts.
4. Dataset generation.
5. Dataset persistence.
6. One forecasting method.
7. Train/test split and metrics.
8. Evaluation persistence.
9. One complete end-to-end vertical slice.

By the end of Day 1, the system should be able to:

- Load a configuration.
- Generate one dataset.
- Persist and retrieve it.
- Execute one forecasting method.
- Calculate metrics.
- Persist and retrieve the evaluation.

This is the first critical milestone.

Day 2 — Complete the research workflow

Priority:

1. Add the other forecasting methods.
2. Add bounded SARIMA selection.
3. Add experiment planning and orchestration.
4. Add progress reporting.
5. Add recovery and resume.
6. Add batch execution.
7. Add grouped analysis.
8. Add CLI workflows.
9. Add export.
10. Add essential integration and research-validity tests.

GUI and advanced visualization are secondary to a reliable research workflow.

If time becomes constrained, prioritize a correct, resumable CLI application over an incomplete GUI.

---

23. Definition of Done

The MVP is complete when all of the following are true.

Research

- □ The generator supports all defined experimental dimensions.
- □ The full design produces 540 reproducible datasets.
- □ Ground-truth components are preserved.
- □ All three forecasting methods work through the common interface.
- □ SARIMA parameter selection is bounded.
- □ The common evaluation protocol is enforced.
- □ Metrics are calculated correctly.

Persistence

- □ Experiments and evaluations are stored in SQLite.
- □ Numerical arrays are stored as NPZ artifacts.
- □ Results can be retrieved after restarting the application.
- □ Artifact integrity is checked.
- □ Backup support exists.

Execution

- □ Individual and batch experiments work.
- □ Existing datasets can be evaluated.
- □ Execution is sequential by default.
- □ Progress is visible.
- □ Individual model failures are isolated.
- □ Interrupted experiments can resume.
- □ Completed evaluations are not repeated unnecessarily.

Analysis

- □ Results can be grouped by experimental dimensions.
- □ Metrics can be compared across methods.
- □ Best methods can be identified per group and metric.
- □ Results can be exported.

Delivery

- □ CLI workflows work.
- □ GUI workflows use shared application services, if included in the MVP.
- □ Linux installation is validated.
- □ Windows installation is validated.
- □ CI passes.
- □ Documentation explains installation, configuration, execution, recovery, and result interpretation.

Research validity

- □ No data leakage.
- □ Fair comparison across methods.
- □ Reproducible data generation.
- □ Correct metrics.
- □ Failed evaluations are reported.
- □ Stored results match the executed experiment.

---

24. Open Implementation Specifications

The architecture is agreed, but some low-level choices must be finalized during implementation rather than silently assumed.

These are implementation specifications, not reasons to reopen the overall architecture.

Item	Required decision
Generator mathematics	Exact trend, seasonal profiles, noise distributions, and parameter ranges
Seed policy	Deterministic mapping from base seed to each dataset replication
Multiplicative data	Positive component and noise-factor constraints
Classical decomposition	Exact trend estimation, seasonal estimation, and extrapolation procedure
Holt-Winters	Exact supported trend/seasonal configurations and initialization policy
SARIMA	Bounded candidate ranges, selection criterion, and failure policy
Forecast horizon	Exact override semantics when configured versus default seasonal cycle
MAPE	Exact handling of zero and near-zero actual values
Retry policy	Which failure categories can be retried automatically or manually
Final experiment status	Exact mapping from evaluation outcomes to experiment status
Database schema	Physical tables, indexes, constraints, and migrations
Artifact format	NPZ field names, schema version, and integrity metadata
CLI interface	Final command names and options
GUI refresh	Concrete mechanism for delivering live progress updates
Supported Python version	Version compatible with selected dependencies and target platforms

Every item must have an explicit decision before the relevant component is considered complete.

---

25. Final Engineering Principle

The system should be built around one central principle:

> Research correctness comes before execution speed, architectural complexity, and interface polish.

A simple system that generates reproducible datasets, applies fair forecasting comparisons, records failures honestly, and preserves every result is more valuable than a sophisticated interface or parallel execution engine that compromises experimental validity.

The first implementation milestone is therefore not the full experiment. It is one complete, verifiable vertical slice:

Configuration → Dataset generation → Persistence → Forecasting → Evaluation → Persistence → Retrieval.

Once that slice works reliably, the remaining capabilities can be added incrementally without losing control of the system's architecture or research methodology.